Sistemas Operativos

¿CÓMO FUNCIONA FUSE POR DENTRO?

Ferrer Cordero José Manuel
Chacón Jaral Hugo Emmanuel

¿Qué es FUSE?

FUSE (Filesystem in Userspace) es un módulo del kernel y marco de trabajo en sistemas tipo Unix (como Linux) que permite a usuarios sin privilegios crear y montar sus propios sistemas de archivos en el espacio de usuario, sin modificar el núcleo. Facilita la creación de sistemas de archivos virtuales, en red o en memoria, mejorando la seguridad y flexibilidad.

Problema que se resuelve →

- Desarrollo en kernel = alta complejidad
- Riesgo de fallos críticos
- Difícil debugging
- Baja flexibilidad para experimentar

Idea Clave

- FUSE actúa como intermediario
- Kernel delega operaciones
- Lógica ejecutada en user space
- Transparente para la aplicación

FUSE actúa como puente entre el kernel y un programa externo.

Arquitectura

1

APLICACIÓN

Comandos ls, cat, cp, editor de textos, navegador o programas de archivos

2

KERNEL

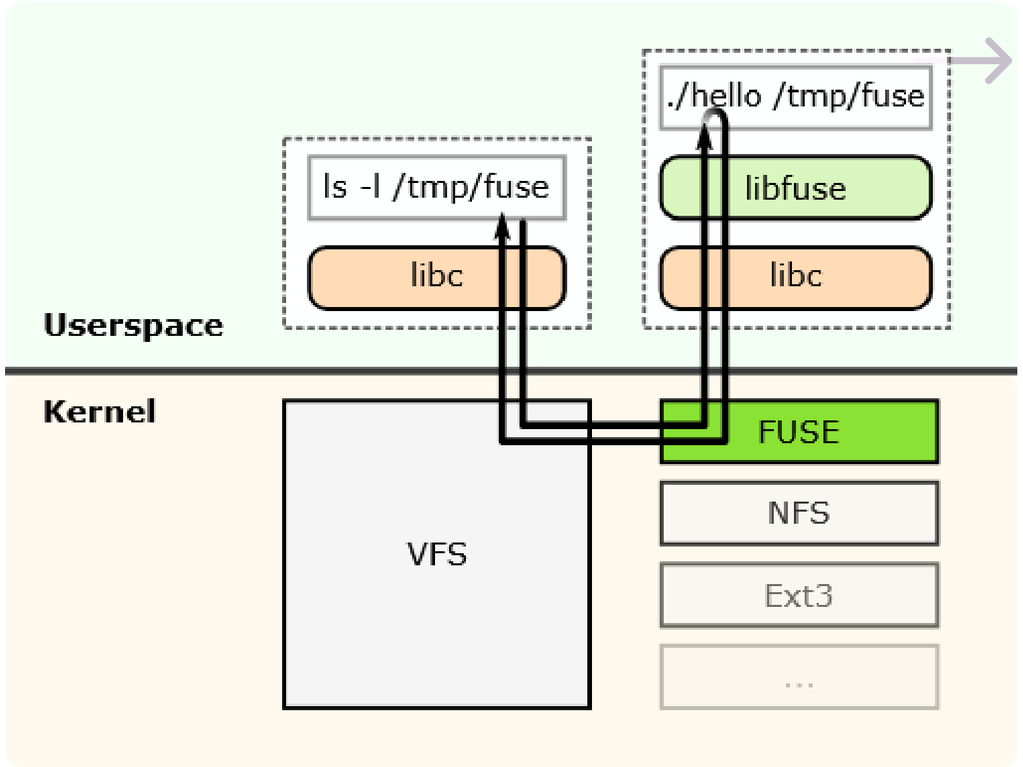
Sistema de archvios virtual de Linux (VFS)

3

DAEMON FUSE

Lógica real

Flujo General



Una operación de archivo viaja del programa al kernel, del kernel al daemon, y regresa.

App pide archivo

↓

Kernel recibe syscall

↓

FUSE la reenvía

↓

Daemon responde

↓

App recibre resultado

Flujo detallado

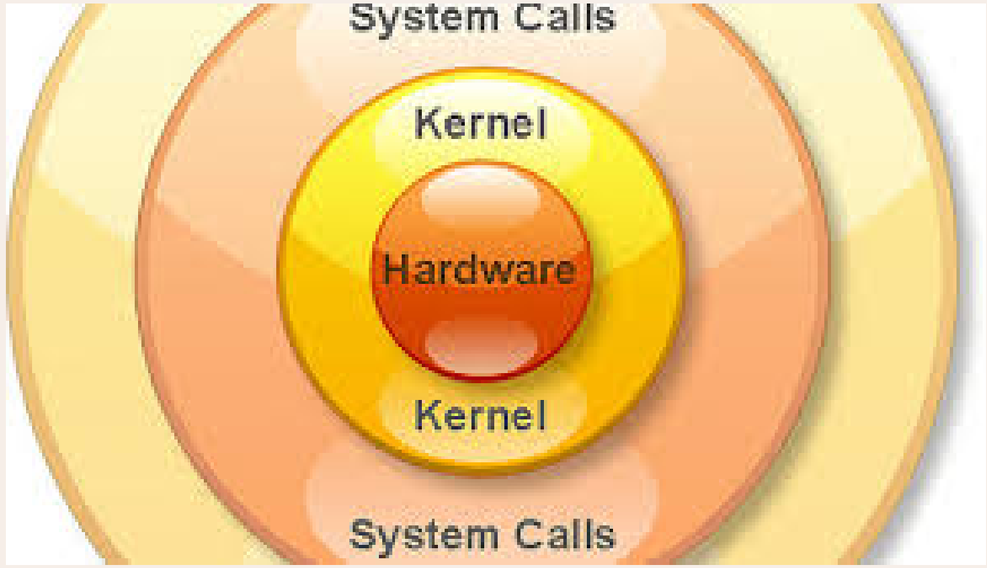
- **cat archivo.txt**
- **open() / read()**
- **VFS del kernel**
- **módulo FUSE**
- **/dev/fuse**
- **daemonFUSE**
- **Respuesta al kernel**
- **Respuesta a la aplicación**

/dev/fuse

Dispositivo especial que conecta el kernel con el daemon en espacio de usuario.

- Kernel
- ↓ ↑
- /dev/fuse
- ↓ ↑
- Deaemon FUSE

Syscalls



open()
Abrir

write()
Escribir

mkdir()
Crear directorio

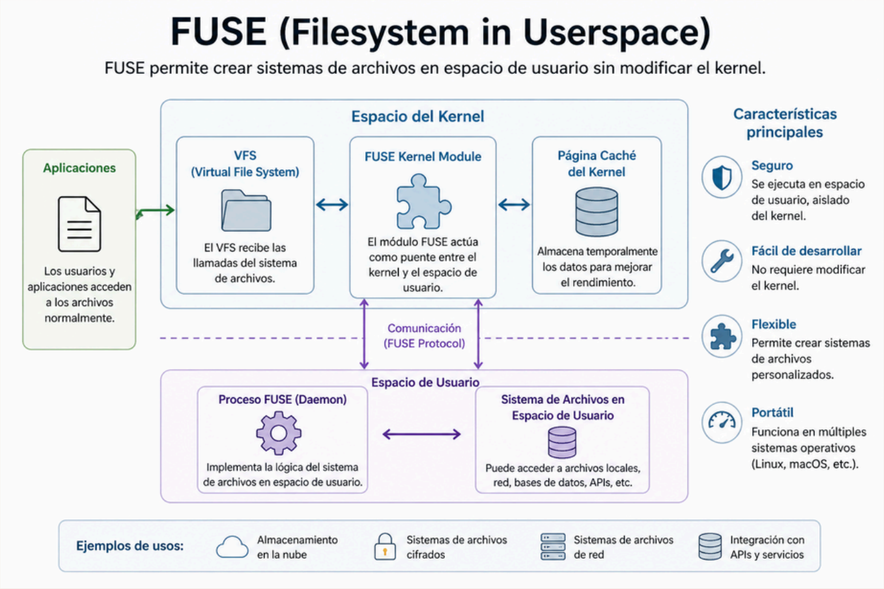
read()
Leer

API de FUSE

La API de FUSE es el conjunto de funciones que el desarrollador implementa para definir el comportamiento del sistema de archivos.
Cuando una aplicación realiza operaciones como open, read o write, el kernel las redirige a FUSE, y este ejecuta las funciones que tú definiste en el daemon.
Las funciones principales incluyen:

- getattr → obtiene atributos del archivo
- readdir → lista directorios
- open, read, write → manejo de archivos
- mkdir, unlink → creación y eliminación

Estas funciones se registran en una estructura que conecta las operaciones del sistema con tu implementación.



Ventajas

Mayor seguridad

Al ejecutarse en espacio de usuario, cualquier error no afecta directamente al kernel.
Esto reduce el riesgo de fallos graves en el sistema operativo.

Facilidad de desarrollo

No es necesario programar en el kernel, lo cual es complejo y peligroso.
Con FUSE, se puede desarrollar en lenguajes comunes como C, Python o Java, haciendo el proceso mucho más accesible.

Flexibilidad

Permite crear sistemas de archivos personalizados fácilmente, como:

- Sistemas cifrados
- Sistemas basados en red
- Integraciones con APIs o servicios externos

Esto lo hace ideal para soluciones específicas.

Desarrollo y depuración más sencilla

Al estar en user space:

- Se pueden usar herramientas normales de depuración
- No es necesario reiniciar el sistema por errores
- Es más rápido probar cambios

Desventajas

Menor rendimiento.

Cada operación no se ejecuta directamente en el kernel, sino que pasa por varios pasos adicionales entre kernel y espacio de usuario.

Mayor latencia

Debido a los cambios de contexto y la comunicación mediante /dev/fuse.

Dependencia del daemon

Ya que si este proceso falla o se vuelve lento, todo el sistema de archivos se ve afectado.

FUSE sacrifica rendimiento a cambio de facilidad y seguridad.
Es una excelente opción para desarrollo rápido, prototipos y sistemas especializados, pero no es ideal cuando se necesitan altas prestaciones y eficiencia máxima.

Comparación: Kernel FS vs FUSE

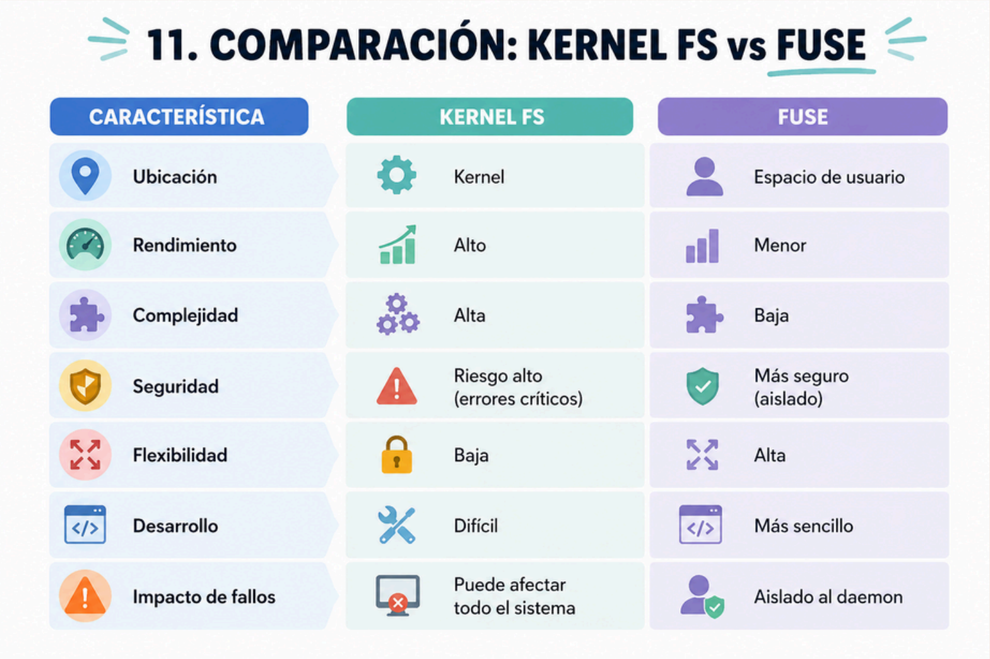
Los sistemas de archivos tradicionales en el kernel y FUSE tienen enfoques muy distintos.

Por un lado, los sistemas de archivos en el kernel ofrecen alto rendimiento, ya que todas las operaciones se ejecutan directamente dentro del sistema operativo, sin intermediarios. Sin embargo, son más complejos de desarrollar y cualquier error puede afectar la estabilidad del sistema completo.

Por otro lado, FUSE permite implementar sistemas de archivos en espacio de usuario, lo que lo hace más fácil, seguro y flexible. No requiere modificar el kernel y los errores no comprometen todo el sistema.

El punto clave es el intercambio:

- Kernel FS → rápido pero complejo y riesgoso
- FUSE → flexible y seguro, pero más lento



CARACTERÍSTICA	KERNEL FS	FUSE
Ubicación	Kernel	Espacio de usuario
Rendimiento	Alto	Menor
Complejidad	Alta	Baja
Seguridad	Riesgo alto (errores críticos)	Más seguro (aislado)
Flexibilidad	Baja	Alta
Desarrollo	Difícil	Más sencillo
Impacto de fallos	Puede afectar todo el sistema	Aislado al daemon

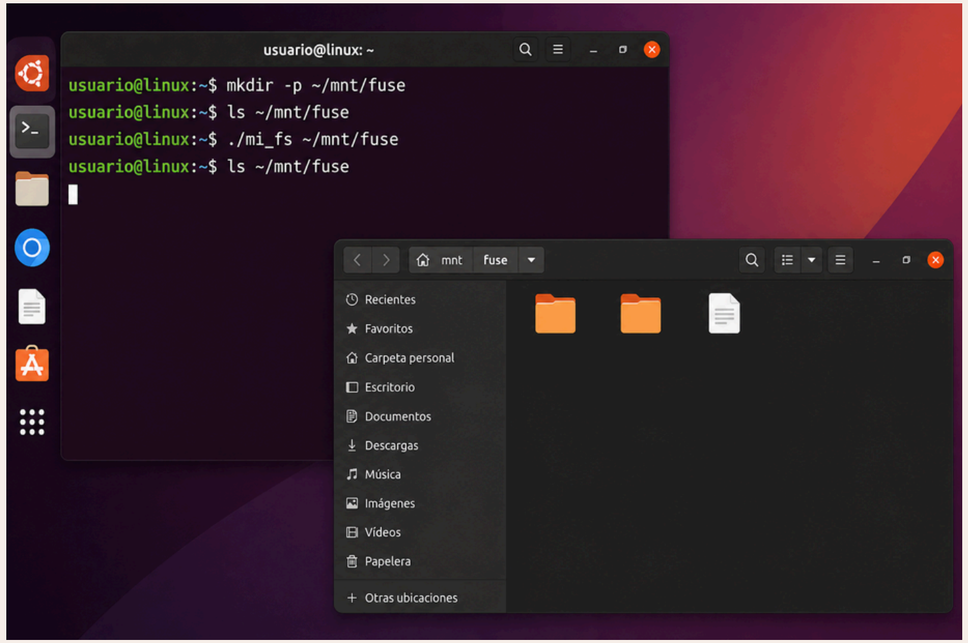
Seguridad

FUSE mejora la seguridad porque traslada la lógica del sistema de archivos al espacio de usuario, en lugar de ejecutarla dentro del kernel.
Esto es importante porque el kernel es una parte crítica del sistema operativo: cualquier error ahí puede provocar fallos graves, cuelgues o incluso vulnerabilidades de seguridad. En cambio, en FUSE, el sistema de archivos se ejecuta como un proceso normal (daemon), por lo que queda aislado del núcleo del sistema.
Gracias a este aislamiento:

- Si el daemon falla, no se cae todo el sistema, solo se ve afectado ese sistema de archivos
- Se reduce el riesgo de corrupción global del sistema
- Es más fácil depurar y corregir errores sin comprometer el sistema operativo
-

Sin embargo, es importante notar que esta seguridad también depende de que el daemon esté bien implementado, ya que sigue siendo un punto vulnerable si no se maneja correctamente

Montaje en FUSE



El montaje en FUSE es el proceso mediante el cual un sistema de archivos desarrollado en espacio de usuario se integra al sistema operativo para poder usarse como cualquier otro sistema de archivos.
Para hacerlo, se ejecuta un comando como:

./mi_fs /mnt/fuse

Aquí:

- ./mi_fs es el programa (daemon) que implementa el sistema de archivos
- /mnt/fuse es el punto de montaje donde se podrá acceder a los archivos

Cuando se monta:

1. El sistema operativo registra ese punto de montaje
2. El módulo FUSE en el kernel se activa
3. Se establece la comunicación con el daemon en user space
4. A partir de ese momento, cualquier acceso a /mnt/fuse se redirige al sistema FUSE

Esto permite que las aplicaciones usen el sistema de archivos como si fuera uno nativo, sin notar que está implementado en espacio de usuario.

Uso ideal

FUSE es más adecuado en escenarios donde se prioriza la flexibilidad, facilidad de desarrollo y seguridad, más que el rendimiento máximo.
Se utiliza principalmente en:

- Prototipos de sistemas de archivos, donde se quiere experimentar sin modificar el kernel
- Sistemas distribuidos o de red, como los que acceden a archivos remotos
- Integración con APIs o servicios externos, por ejemplo, montar datos de la nube como si fueran archivos locales
- Sistemas cifrados, donde se añade una capa de seguridad sobre archivos existentes

Esto es posible porque FUSE permite implementar lógica compleja en user space de forma rápida y segura.
En estos casos, la ligera pérdida de rendimiento es aceptable frente a los beneficios que ofrece.

FUSE no es recomendado

FUSE no es la mejor opción en escenarios donde se requiere alto rendimiento, baja latencia o máxima eficiencia.
Esto incluye:

- Sistemas críticos, donde cualquier retraso puede afectar el funcionamiento general
- Bases de datos de alto rendimiento, que necesitan acceso rápido y constante a disco
- Aplicaciones en tiempo real, donde cada milisegundo cuenta
- Sistemas con gran volumen de I/O, como servidores de almacenamiento intensivo

La razón principal es que FUSE introduce overhead y latencia debido a la comunicación constante entre kernel y espacio de usuario.
Además, al depender de un daemon, se añade un posible punto de falla adicional, lo cual no es ideal en sistemas donde se requiere máxima estabilidad.

REFERENCIAS

1. Bovet, D. P., & Cesati, M. (2006). Entendiendo el núcleo de Linux. Madrid, España: Anaya Multimedia.
2. Gómez, E., & Martínez, J. (2021). Sistemas de archivos en espacio de usuario mediante FUSE. Revista Iberoamericana de Tecnologías del Software, 8(2), 45–53.
3. Silberschatz, A., Galvin, P. B., & Gagne, G. (2015). Fundamentos de sistemas operativos (9.ª ed.). México: McGraw-Hill.
4. Tanenbaum, A. S., & Bos, H. (2016). Sistemas operativos modernos (4.ª ed.). México: Pearson Educación.
5. Bach, M. J. (1990). Diseño del sistema operativo UNIX. México: Prentice Hall.